	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Prácticas(s): Proyecto02

Integrante(s): 323078401

323115296

323082392

323139441

323253916

No. de lista o brigada: 04

Semestre: 2027-I

Fecha de entrega: 25/10/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**

1.1. Planteamiento del problema 2

1.2. Motivación 2

1.3. Objetivos 2

2. Marco Teórico **3**

3. Desarrollo **4**

3.1. Cambios relevantes 4

3.2. Funcionamiento 5

4. Resultados **9**

5. Conclusiones **10**

6. Referencias **11**

1. Introducción

1.1 Planteamiento del problema

Se busca dar continuidad al simulador del sistema hospitalario. En esta segunda etapa se necesita distinguir entre diferentes categorías de pacientes, atención general y de especialidad; y diversos perfiles de personal clínico y directivo: médicos generales, médicos especialistas, enfermeros especialistas y jefatura de enfermería. Al mismo tiempo, necesita reestructurarse mediante clases abstractas, herencia, interfaces, polimorfismo, paquetes y encapsulamiento preservando las reglas operativas del hospital.

1.2 Motivación

La incorporación de múltiples categorías de personal de salud y de pacientes dentro del mismo sistema hospitalario. Estructurar el programa mediante jerarquías de herencia y contratos definidos por interfaces. Asimismo, proteger el estado interno de los objetos mediante modificadores de acceso y validaciones para evitar que datos sensibles, como la cédula profesional de un médico, la identidad de un paciente o los expedientes disciplinarios, sufran alteraciones inválidas desde clases externas a lo largo de la simulación.

1.3 Objetivos

- Aplicar el principio de encapsulamiento en las clases del sistema mediante el uso de modificadores de acceso privados y métodos de acceso o modificación acordes con la mutabilidad de cada atributo.
- Organizar las clases e interfaces del simulador en paquetes de Java, empleando declaraciones package, sentencias import y *Fully Qualified Names*.
- Modelar las jerarquías de herencia para pacientes, médicos y enfermeros a partir de clases base abstractas y subclases especializadas.
- Definir e implementar la interfaz AtencionMedica como contrato común para el personal médico y de enfermería.
- Implementar mecanismos de polimorfismo estático y polimorfismo dinámico en las tres jerarquías del sistema.

2. Marco Teórico

El encapsulamiento consiste en ocultar el estado interno de un objeto y exigir que toda interacción con sus datos se realice a través de sus propios métodos [1]. En Java, el control de acceso se establece mediante modificadores de visibilidad como private, el cual restringe el acceso de un atributo o método exclusivamente a la clase donde fue declarado [2]. Cuando este modificador se combina con la palabra reservada final en la declaración de un atributo, su valor solo puede asignarse una vez dentro del constructor, garantizando su inmutabilidad posterior y prescindiendo de métodos modificadores (*setters*), mientras que los atributos cuyo estado debe cambiar durante el ciclo de vida del objeto exponen métodos de consulta (*getters*) y de modificación (*setters*) encargados de validar que no ingresen valores nulos o fuera de rango [1, 2].

Para estructurar proyectos compuestos por múltiples clases relacionadas, Java utiliza paquetes (*packages*), los cuales agrupan tipos funcionalmente con el mismo fin en espacios de nombres diferenciados que previenen colisiones de identificadores [3]. Todo archivo fuente que pertenece a un paquete incluye la directiva package al inicio del documento y, para interactuar con clases públicas alojadas en otros paquetes, recurre a la directiva import o bien invoca directamente al elemento mediante su *Fully Qualified Name*, formado por la ruta del paquete seguida del identificador de la clase [3].

Por su parte, la herencia permite definir nuevas clases derivadas a partir de clases base existentes mediante la palabra reservada extends, heredando sus atributos y comportamientos accesibles para reutilizar código y añadir especializaciones propias [4]. Dentro de una subclase, la palabra reservada super se utiliza tanto para invocar al constructor de la clase padre durante la inicialización como para ejecutar la versión original de un método heredado antes de agregar lógica adicional [4]. Cuando una clase base representa un concepto general que no debe instanciarse directamente por sí mismo, se declara con el modificador abstract [5]. Una clase abstracta puede concentrar atributos y métodos concretos compartidos, además de declarar métodos abstractos, sin cuerpo, que obligan a las subclases concretas a proporcionar su propia implementación [5]. De forma complementaria, una interfaz (interface) establece un contrato de métodos abstractos que pueden ser implementados por clases pertenecientes a jerarquías distintas mediante implements [6].

El polimorfismo es entendido como la capacidad de que una operación o referencia adopte múltiples comportamientos según el contexto [4, 7]. Además, existe el polimorfismo estático y el polimorfismo dinámico. El polimorfismo estático se manifiesta en Java mediante la sobrecarga de métodos y constructores, donde varias definiciones comparten el mismo nombre dentro de una clase o interfaz pero difieren en el número o tipo de sus parámetros, de modo que el compilador determina cuál enlazar en tiempo

de compilación [1]. En contraste, el polimorfismo dinámico se fundamenta en la sobrescritura de métodos heredados y en el enlace tardío: una variable o arreglo declarado con el tipo de una superclase abstracta o de una interfaz puede almacenar referencias a instancias de cualquier subclase concreta, y al invocar un método sobrescrito sobre dicha referencia, la Máquina Virtual de Java resuelve y ejecuta en tiempo de ejecución la implementación que corresponde al tipo real del objeto en memoria [4, 7].

3. Desarrollo

3.1 Cambios relevantes

El programa de la segunda etapa conserva la funcionalidad de simulación hospitalaria de la versión inicial, pero transforma su arquitectura interna al pasar de cinco clases alojadas en el paquete por defecto a un sistema integrado por trece archivos de código fuente distribuidos en cinco paquetes de Java. El paquete interfaces aloja la interfaz AtencionMedica, aislando el contrato de atención clínica del resto de la implementación; el paquete hospital.pacientes agrupa a la clase abstracta Paciente junto con sus subclases PacienteGeneral y PacienteEspecialidad; el paquete hospital.personal contiene a las clases abstractas Medico y Enfermero, así como a las clases derivadas MedicoGeneral, MedicoEspecialista, EnfermeroEspecialista y JefeEnfermeria; el paquete hospital.sistema concentra al controlador Sistema, a la clase adicional ActaAdministrativa y a la interfaz de consola Main; y finalmente el paquete test incorpora la suite de verificación SistemaHospitalarioTest.

La comunicación entre estos paquetes se formaliza mediante sentencias import al inicio de cada archivo. Además, el código hace uso explícito de *Fully Qualified Names* en dos puntos: dentro de la clase JefeEnfermeria, donde no se importa la clase ActaAdministrativa con import, sino que el parámetro del método gestionarActaAdministrativa se declara directamente con su ruta completa hospital.sistema.ActaAdministrativa; y en el método main de la clase Main, donde el lector de teclado se declara e instancia mediante java.util.Scanner.

En todas las clases del dominio se aplica encapsulamiento declarando la totalidad de los atributos de instancia con visibilidad private. En Paciente, el atributo nombre se declara private final; del mismo modo, en Medico y Enfermero los atributos nombre, cedula y especialidad son private final y solo tienen métodos get. En la clase ActaAdministrativa, el folio, el pacienteAfectado, el enfermeroReportado, la fechaHora, el motivo y la descripcionHechos también se definen como private final para garantizar que los hechos asentados en una queja no puedan alterarse una vez levantado el reporte. En con-

traparte, los atributos cuyo valor evoluciona durante la operación, como: el tratamiento, las banderas booleanas de estado y la especialidad en Paciente, el motivoConsulta en PacienteGeneral, el nivelUrgencia y diagnosticoPrevio en PacienteEspecialidad, el consultorioAsignado en MedicoGeneral, los aniosExperiencia y subespecialidad en MedicoEspecialista, la certificacionAvanzada en EnfermeroEspecialista, y el turnoSupervision y pisoAsignado en JefeEnfermeria; cuentan con métodos set dotados de validaciones.

Dichas validaciones: utilizan los métodos trim() e isEmpty() para impedir la asignación de referencias nulas o cadenas en blanco sustituyéndolas por valores seguros por defecto, normalizando especialidades y turnos a mayúsculas con toUpperCase(), acotan los rangos numéricos en el nivel de urgencia para que permanezca siempre entre 1 y 5, se aseguran de que los años de experiencia no sean negativos y el piso asignado sea como mínimo 1. De manera adicional, Medico y Enfermero protegen sus arreglos internos de pacientes asignados omitiendo un método set para el arreglo o para el contador noPacientes, y devolviendo en getPacientesAsignados() una copia defensiva del arreglo para evitar modificaciones externas directas.

3.2 Funcionamiento

La clase Paciente se rediseñó como una clase abstracta que conserva sus cinco atributos privados, la inicialización en su constructor, el método de registro en el sistema y el método verTratamiento(), pero incorpora dos métodos abstractos: solicitarConsulta(Sistema) y obtenerTipoCondicion(). Al declarar Paciente como abstracta, se impide la creación de pacientes sin clasificación clínica. Los métodos concretos verTratamiento() y toString() de la clase base invocan internamente al método abstracto obtenerTipoCondicion(), delegando en las subclases el detalle descriptivo de cada paciente.

De esta clase base extienden PacienteGeneral y PacienteEspecialidad. La subclase PacienteGeneral modela a los pacientes de atención primaria; su constructor recibe el nombre y el motivo de consulta, invoca al constructor de la superclase para fijar automáticamente su especialidad inicial como general, e inicializa su atributo propio motivoConsulta. Además incorpora el comportamiento propio actualizarMotivo(String), sobrescribe obtenerTipoCondicion() para mostrar su motivo y redefine solicitarConsulta(Sistema) imprimiendo el turno de medicina general antes de pedir al sistema su asignación automática.

Por su parte, PacienteEspecialidad representa a quienes requieren atención en una rama específica; invoca al constructor de la superclase e incorpora dos atributos propios: nivelUrgencia (escala de triaje del 1 al 5) y diagnosticoPrevio, junto con el comportamiento propio actualizarTriage(int, String). Esta subclase sobrescribe solicitarConsulta(Sistema) detallando en consola su prioridad de triaje, la especialidad

demandada y su diagnóstico preliminar antes de invocar la asignación automática del controlador.

Para estandarizar las acciones clínicas de los profesionales de la salud, la interfaz AtencionMedica declara tres métodos: darConsulta(Paciente), darTratamiento(Paciente) y una versión sobrecargada darTratamiento(Paciente, String) que recibe la indicación médica explícita. La clase abstracta Medico implementa esta interfaz y concentra la estructura común de los médicos: sus atributos inmutables (nombre, cedula, especialidad), el contador noPacientes, el arreglo pacientesAsignados limitado a 10 pacientes maximo y la referencia pacienteEnConsulta, la cual conserva la restricción de permitir únicamente un paciente en consulta activa a la vez. Asimismo, Medico declara los métodos abstractos puedeAtender(Paciente) y getTipoMedico(), de modo que sus métodos concretos admitirPaciente(Paciente) y solicitarPaciente(Sistema) verifican la compatibilidad invocando polimórficamente a puedeAtender(Paciente) sin conocer de antemano de qué subclase de médico se trata.

Cuando se invoca darConsulta(Paciente) en Medico, el método comprueba que el paciente no sea nulo y que el paciente en consulta sea nulo o apunte al mismo paciente; si otro paciente ya está siendo atendido, rechaza la operación, y si el médico está libre, vincula la referencia en pacienteEnConsulta y actualiza las banderas del paciente a consulta activa. En darTratamiento(Paciente, String), si el paciente recibido coincide con pacienteEnConsulta, el médico libera su puntero asignándole null, cambia las banderas del paciente de consulta a tratamiento y guarda la receta; mientras que la versión sobrecargada de un solo parámetro darTratamiento(Paciente) reutiliza la anterior enviando una indicación de reposo general por defecto. Adicionalmente, el método verListaPacientes() ahora crea una copia local de los pacientes asignados y ejecuta los métodos privados recursivos mergeSortDescendente y mezclarDescendente, comparando los nombres completos con compareToIgnoreCase(...) ≥ 0 para imprimirlos de la Z a la A sin alterar el orden de llegada en el arreglo original.

Las subclases concretas MedicoGeneral y MedicoEspecialista extienden a Medico. MedicoGeneral fija su especialidad en “GENERAL” mediante super del constructor, añade el atributo propio consultorioAsignado y sobrescribe puedeAtender(Paciente) devolviendo verdadero sólo cuando la especialidad del paciente es “GENERAL”. En su sobrescritura de darConsulta(Paciente), invoca primero a super para ejecutar el cambio de estado base y agrega la impresión de la revisión primaria en su consultorio; además, define el método propio derivarAEspecialista(Paciente, String), que permite modificar la especialidad de un paciente general para canalizarlo hacia un especialista.

Por otro lado, MedicoEspecialista invoca super del constructor, incorpora los atributos propios aniosExperiencia y subespecialidad, y sobrescribe puedeAtender(Paciente) exigiendo coincidencia exacta con equalsIgnoreCase entre su especialidad y la del pa-

ciente. También sobrescribe darConsulta(Paciente) llamando a super e inspeccionando mediante instanceof si el argumento es una instancia de PacienteEspecialidad para mostrar su nivel de urgencia y diagnóstico previo; sobrescribe darTratamiento(Paciente, String) anteponiendo el prefijo del protocolo especializado a través de super; e implementa el comportamiento propio emitirDictamenEspecializado(Paciente, String).

De forma análoga a la rama médica, la clase abstracta Enfermero implementa la interfaz AtencionMedica, encapsula sus credenciales inmutables y administra un arreglo pacientesAsignados restringido a máximo 3 pacientes y aplicando también el algoritmo *Merge Sort* descendente en verListaPacientes(). La clase Enfermero define tanto darConsulta(Paciente), orientado a la toma de signos vitales y valoración de enfermería, como las dos versiones sobrecargadas de darTratamiento, donde la versión de un argumento administra el tratamiento que el paciente ya tiene prescrito y la versión de dos argumentos registra un cuidado específico. También define los métodos abstractos puedeAtender(Paciente) y getCargo(), utilizados por admitirPaciente(Paciente) y toString().

De Enfermero derivan EnfermeroEspecialista y JefeEnfermeria. EnfermeroEspecialista inicializa su clase base con super del constructor, agrega el atributo propio certificacionAvanzada, implementa el comportamiento propio monitoreoIntensivo(Paciente), restringe puedeAtender(Paciente) únicamente a pacientes cuya especialidad coincida con la suya y sobrescribe darTratamiento(Paciente) apoyándose en el super para detallar el uso de su certificación avanzada.

Por su parte, JefeEnfermeria modela el rol directivo y de coordinación dentro de la jerarquía. Su constructor invoca super del constructor con la especialidad general e inicializa los atributos propios turnoSupervision y pisoAsignado. Sobrescribe puedeAtender(Paciente) devolviendo verdadero para cualquier referencia no nula, lo que permite a la jefatura para asistir a pacientes de cualquier especialidad, y sobrescribe darConsulta(Paciente) invocando al super para validar el plan de cuidados desde la jefatura. Para cumplir y ampliar el requisito de contar con funciones adicionales de supervisión o coordinación, JefeEnfermeria implementa dos operaciones exclusivas: supervisarGuardia(Enfermero[], int), que recorre el arreglo de enfermeros del hospital auditando el cargo polimórfico y la ocupación de cupo de cada elemento, y gestionarActaAdministrativa(ActaAdministrativa, String, String). Esta segunda función colabora estrechamente con la clase adicional ActaAdministrativa, la cual genera un número de folio consecutivo mediante un atributo contadorFolios cada vez que un paciente levanta una queja contra un enfermero, y cambia su estado a procesada cuando el jefe de enfermería emite un dictamen y la canaliza hacia Recursos Humanos, el Comité de Ética o el Consejo Directivo.

La clase Sistema funge como el controlador central que integra todas las jerarquías

sin crear estructuras de datos separadas para cada subclase concreta. Para ello, declara los arreglos polimórficos Medico[] medicos con capacidad inicial de 50, Enfermero[] enfermeros con capacidad inicial de 100, Paciente[] pacientes con capacidad inicial de 10 y ActaAdministrativa[] actasAdministrativas con capacidad inicial de 20. Los métodos registrarMedico, registrarEnfermero, registrarPaciente y registrarActa verifican la ausencia de duplicados y duplican automáticamente la capacidad de cualquiera de los cuatro arreglos (length * 2) cuando se llenan.

El polimorfismo estático dentro de Sistema se aplica tanto en sus dos constructores sobrecargados, uno que recibe el nombre del hospital y otro por defecto que invoca al primero mediante this("Hospital Central Universitario"), como en el método sobrecargado asignarPaciente, el cual presenta una versión (Paciente, Medico) y otra (Paciente, Enfermero). En ambos métodos, así como en asignarPacienteAutomatico(Paciente), se manifiesta el polimorfismo dinámico: Sistema recorre sus arreglos de supertipo e invoca puedeAtender(paciente) y admitirPaciente(paciente) sobre referencias de tipo Medico o Enfermero, delegando en el enlace dinámico de Java la validación de si se trata de un profesional general, especialista o jefe de enfermería.

La clase Main controla la ejecución interactiva mediante menús por consola operados con Scanner, cargando inicialmente un conjunto de objetos de prueba de todas las subclases concretas. En los submenús de médicos, enfermeros y pacientes, las selecciones devuelven referencias de las clases abstractas Medico, Enfermero y Paciente, sobre las cuales se ejecutan de forma polimórfica las solicitudes de consulta, las consultas médicas, la prescripción o administración de tratamientos y el listado ordenado con *Merge Sort*, recurriendo a la verificación con instanceof y conversión descendente únicamente cuando se solicita una operación exclusiva de una subclase, como emitir un dictamen especializado en MedicoEspecialista o supervisar la guardia y dictaminar actas en JefeEnfermeria. Adicionalmente, Main incluye el método demostrarPolimorfismo(Sistema), diseñado para exhibir en un entorno controlado: el despacho dinámico sobre un arreglo de la interfaz AtencionMedica[], que agrupa simultáneamente a un MedicoGeneral, un MedicoEspecialista y un EnfermeroEspecialista, el despacho dinámico sobre un arreglo Paciente[] y la invocación de las dos versiones sobrecargadas de darTratamiento. Finalmente, la clase SistemaHospitalarioTest complementa el desarrollo mediante seis métodos de prueba unitaria automatizada que evalúan la inicialización de atributos inmutables y valores por defecto, el rechazo al superar los cupos de 10 y 3 pacientes, la compatibilidad de especialidades, el polimorfismo a través de AtencionMedica, la supervisión de JefeEnfermeria y el ordenamiento descendente con *Merge Sort*.

4. Resultados

```
Médico General Dr./Dra. DR. ROBERTO SALAS registrado exitosamente.
Médico Especialista en CARDIOLOGIA Dr./Dra. DRA. CLARA MENDEZ registrado exitosamente.
Médico Especialista en PEDIATRIA Dr./Dra. DR. MIGUEL TORRES registrado exitosamente.
Enfermero Especialista en CARDIOLOGIA LIC. ANA LAURA registrado exitosamente.
 controlada)).
Paciente MARIA GARCIA registrado en el sistema (Especialidad: CARDIOLOGIA (Prioridad 4/5 - Arritmia sinusal severa)).
Paciente LUCIA RAMOS registrado en el sistema (Especialidad: PEDIATRIA (Prioridad 2/5 - Revisión mensual de crecimiento)).
```

Figura 1: Captura que muestre la estructura de paquetes del proyecto y la compilación/ejecución inicial del simulador en la Etapa 2.

```
--- 1. POLIMORFISMO MEDIANTE LA INTERFAZ AtencionMedica ---
Ejecutando darConsulta() en objeto de tipo real: MedicoGeneral
El Dr./Dra. DRA. GABRIELA RUIZ está dando consulta médica a: VALENTINA LOPEZ
  [Atención Primaria en Consultorio 102] El Dr./Dra. DRA. GABRIELA RUIZ realiza revisión general de signos vitales e historial clíni
o.
Ejecutando darConsulta() en objeto de tipo real: MedicoEspecialista
El Dr./Dra. DR. HÉCTOR VEGA está dando consulta médica a: VALENTINA LOPEZ
  [Consulta de Alta Especialidad en CARDIOLOGIA (Hemodinamia)]
  Dr./Dra. DR. HÉCTOR VEGA (15 años de exp.) realiza evaluación diagnóstica avanzada.
  Revisando caso con urgencia nivel 4/5 y diagnóstico previo: Dolor torácico
Ejecutando darConsulta() en objeto de tipo real: EnfermeroEspecialista
El personal de enfermería LIC. CARMEN SOSA toma signos vitales y evalúa a: VALENTINA LOPEZ

--- 2. POLIMORFISMO EN JERARQUÍA DE PACIENTES ---
Médico General Dr./Dra. DR. DEMO GENERAL registrado exitosamente.
Médico Especialista en PEDIATRIA Dr./Dra. DRA. DEMO PEDIATRIA registrado exitosamente.

Invocando solicitarConsulta() polimórfico en: PacienteGeneral
El paciente general ALBERTO DIAZ solicita turno de medicina general por: Chequeo anual preventivo
Paciente ALBERTO DIAZ admitido con el Dr./Dra. DR. DEMO GENERAL (1/10).

Invocando solicitarConsulta() polimórfico en: PacienteEspecialidad
El paciente de especialidad BEATRIZ MORA [Prioridad 3/5] solicita especialista en PEDIATRIA por diagnóstico: Fiebre persistente
Paciente BEATRIZ MORA admitido con el Dr./Dra. DRA. DEMO PEDIATRIA (1/10).

--- 3. POLIMORFISMO ESTÁTICO (SOBRECARGA DE MÉTODOS) ---
Llamando a darTratamiento(paciente) [1 argumento]:
El Dr./Dra. DR. ROBERTO SALAS recetó e indicó a VALENTINA LOPEZ: Reposo y seguimiento clínico general.
Llamando a darTratamiento(paciente, indicacion) [2 argumentos]:
El Dr./Dra. DR. ROBERTO SALAS recetó e indicó a VALENTINA LOPEZ: Enalapril 10mg cada 12 horas por 30 días.
```

Figura 2: Captura que muestre la ejecución de la demostración de polimorfismo dinámico y polimorfismo estático.

```
------------------------------------------------------------
           ACTA ADMINISTRATIVA HOSPITALARIA
------------------------------------------------------------
Folio: ACTA-1001 | Fecha/Hora: 25/10/2026
Paciente Denunciante: JUAN PEREZ
Personal Reportado:   LIC. ANA LAURA (Enfermero Especialista en CARDIOLOGIA)
Cédula Profesional:   40404
Especialidad:         CARDIOLOGIA
Motivo del reporte:   LAENFERMERA ANA LAURA COMETIO INTENTO DE VENTA DE MEDICAMENTOS EXTERNOS A LO CITADO POR EL DOCTOR DE CARDIOLOGIA
Descripción Hechos:   Laenfermera ana laura cometio intento de venta de medicame
Estado del trámite:   PENDIENTE DE REVISIÓN
------------------------------------------------------------
```

Figura 3: Captura de las funciones exclusivas de JefeEnfermeria: supervisión de guardia y dictaminación de un ActaAdministrativa.

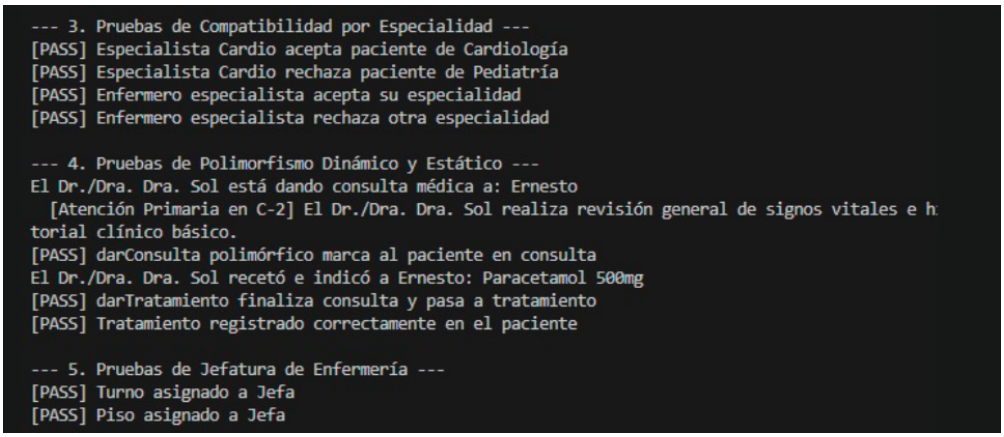


Figura 4: Captura de la salida de SistemaHospitalarioTest.

Ver imagen diagrama_clases_uml_p2.png en GitHub

Figura 5: Diagrama de clases UML de la Etapa 2.

5. Conclusiones

El uso de tres jerarquías de clases abstractas en Paciente, Medico y Enfermero; y la interfaz AtencionMedica demostró su utilidad al permitir ampliar las capacidades del simulador hospitalario. El encapsulamiento con la distinción entre atributos declarados como private final y atributos privados con validaciones en sus *setters* resultó útil para proteger en el código la integridad de datos fijos como cédulas profesionales, nombres y folios de actas administrativas.

Asimismo, el uso de la herencia y la palabra reservada super evitó duplicar dentro de cada jerarquía los atributos de identificación, el control de cupos máximos. La implementación del polimorfismo dinámico en los métodos sobrescritos, combinada con el polimorfismo estático en la sobrecarga de algunos métodos, nos permitió que la clase Sistema gestione en arreglos únicos de supertipo a pacientes generales y de especialidad, así como a médicos generales, especialistas, enfermeros especialistas y jefes de enfermería, lo que facilitó su procesamiento.

6. Referencias

[1] Oracle, "Declaring Member Variables" and "Defining Methods," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html

[2] Oracle, "Controlling Access to Members of a Class," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html

[3] Oracle, "Creating and Using Packages," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/package/packages.html

[4] Oracle, "Inheritance" and "Using the Keyword super," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/IandI/subclasses.html

[5] Oracle, "Abstract Methods and Classes," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html

[6] Oracle, "Interfaces," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html

[7] Oracle, "Polymorphism," *The Java Tutorials*, 2022. [Online]. Available: https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html